SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5116

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Assessment Tool Weightage : 20%

QUESTIONS: **Total Marks: 50**

Annexure A

Scenario-Based

Code of Conduct:

I Vempati Murale Karrtik (192312567) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A
Scenario-based

1. Secure Data Transmission using Block Cipher
A company is transmitting repeated financial data blocks over a network. Initially, they used ECB mode and noticed patterns in ciphertext.

A. Explain why this issue occurs.
B. Suggest a more secure block cipher mode and justify your choice based on the scenario.

2. Choosing an Encryption Algorithm for Banking System
A banking application needs high security and moderate performance for encrypting customer transactions. The available options are DES, 3-DES, and Blowfish.

A. Identify the most suitable algorithm for this scenario.
B. Justify your choice based on security strength and efficiency.

3. Key Exchange in an Unsecured Network
Two users want to establish a secure communication channel over a public network using Diffie-Hellman key exchange. However, the network is vulnerable to attacks.

A. Explain how the key exchange works in this scenario.
B. Identify a possible security threat and suggest a solution to mitigate it.

4. Public Key Cryptography for Email Security
An organization plans to implement RSA for secure email communication.

A. Describe how RSA can be used for encryption and decryption in this scenario.
B. Explain how key distribution and management play a role in maintaining security.

5. Cryptography for IoT Devices
A smart home system uses low-power IoT devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC).

A. Analyze the constraints of IoT devices.
B. Recommend a suitable cryptographic approach and justify your answer.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Annexure A: Scenario-based Questions 2 and 5

2. Choosing an Encryption Algorithm for Banking System

Understanding of the Scenario

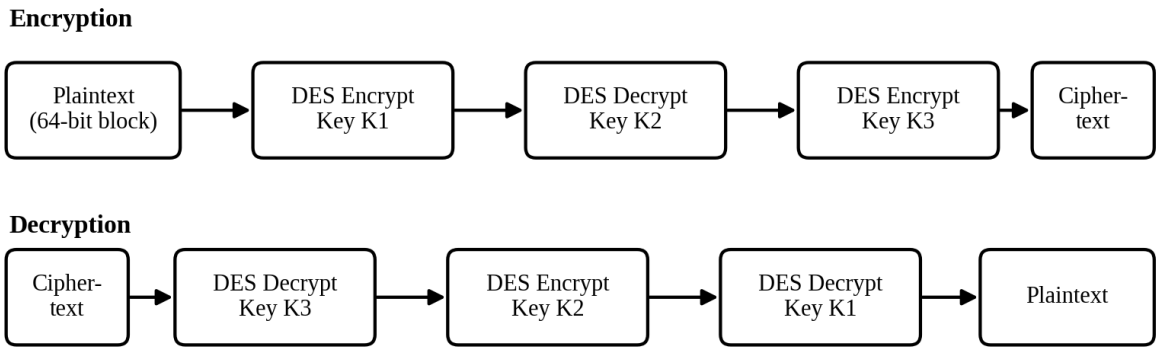
A banking application must encrypt customer transactions, so confidentiality of financial data is the main requirement. The system needs high security but only moderate performance, which means a small amount of extra processing time is acceptable as long as the protection is strong. The three candidates are DES, 3-DES and Blowfish. All three are symmetric block ciphers that work on 64-bit blocks, so the decision depends mainly on key strength, known attacks, standardisation and speed.

A. Identify the most suitable algorithm for this scenario

Answer: 3-DES (Triple DES) is the most suitable algorithm among the three options.

3-DES applies the DES algorithm three times to every 64-bit block using the Encrypt-Decrypt-Encrypt (EDE) sequence with keys K1, K2 and K3. The first stage encrypts the block with K1, the second stage decrypts the result with K2, and the third stage encrypts again with K3. Decryption simply reverses the order. This repeated processing multiplies the key length and removes the main weakness of single DES while still using the proven DES structure. Figure 1 shows the process.

Figure 1: 3-DES (EDE) Encryption and Decryption Process



DES is rejected because its 56-bit key is too short for financial data. Blowfish is rejected because, although it is fast and flexible, it is not a standardised or certified algorithm for banking and its key setup is slow. 3-DES gives the best balance of strong security, wide acceptance in the banking industry and acceptable speed.

B. Justify your choice based on security strength and efficiency

1. Security strength

- **DES:** The key is only 56 bits long, giving about 7.2 x 10^16 possible keys. In 1998 a purpose-built machine broke a DES key in under three days, and modern hardware does it far faster. DES is therefore unsafe for customer transactions.

- **3-DES:** Three independent 56-bit keys give a 168-bit key. Because of the meet-in-the-middle attack the effective strength is about 112 bits, which is still far beyond brute-force capability. 3-DES with K1 = K2 = K3 behaves like single DES, so it stays compatible with existing banking hardware such as ATM networks, payment terminals and hardware security modules.
- **Blowfish:** The key can be 32 to 448 bits and no practical attack on the full 16-round cipher is known. However, it is not approved by standards bodies such as NIST, which makes compliance audits in banking difficult. Its 64-bit block size also makes it open to birthday-type attacks (Sweet32) when very large amounts of data are encrypted under one key.

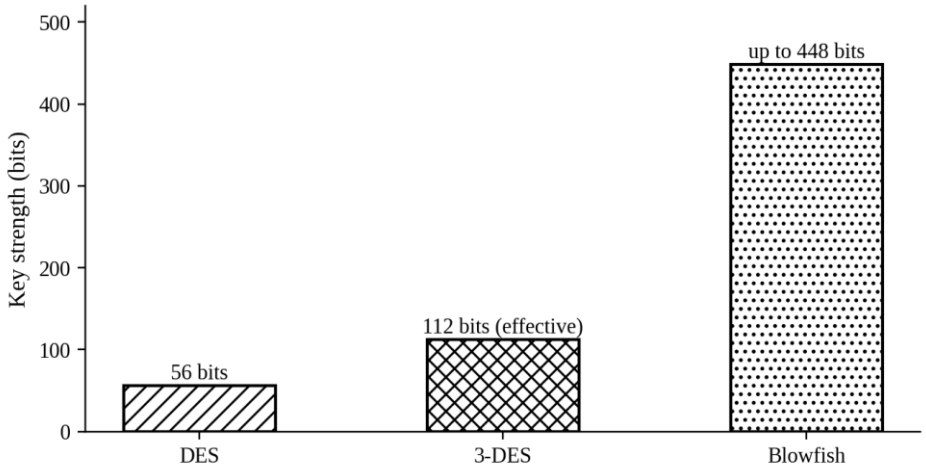
2. Efficiency

- **DES:** Fastest of the three because it makes one pass of 16 rounds, but this speed is meaningless when the security is broken.
- **3-DES:** It performs 48 rounds, so it is about three times slower than DES. Transaction records are short messages, and the scenario states that only moderate performance is needed, so this overhead is acceptable. Hardware support for 3-DES is also widely available.
- **Blowfish:** Very fast in software once the key is set, but the key schedule is expensive because it generates about 4 KB of subkey tables. This is a disadvantage when keys change often, for example per session or per transaction.

3. Comparison of the algorithms

Feature	**DES**	**3-DES**	**Blowfish**
Block size	64 bits	64 bits	64 bits
Key size	56 bits	112 or 168 bits (112 effective)	32 to 448 bits
Number of rounds	16	48 (3 x 16)	16
Resistance to brute force	Very weak	Strong	Strong
Speed	Fast	Moderate (about 3 times slower than DES)	Very fast after key setup
Key setup	Fast	Moderate	Slow
Main weakness	Short key	Small block size, meet-in-the-middle	Small block size, not standardised
Suitability for banking	Not suitable	Most suitable	Acceptable but less preferred

Figure 2: Key Length Comparison (64-bit block size for all three)



Conclusion: 3-DES is the best choice for the banking system because it offers strong protection, is accepted by financial standards and fits the requirement of moderate performance. For safe use, three independent keys should be used, the cipher should run in CBC mode with a random IV, keys should be changed regularly so that not too much data is encrypted under one key, and the keys should be stored in a hardware security module. For new banking systems, AES is the modern replacement and migration to it should be planned.

5. Cryptography for IoT Devices

Understanding of the Scenario

A smart home system contains many small, low-power devices such as door locks, motion sensors, smart bulbs and thermostats. These devices send private data (for example whether a person is at home) and receive control commands, so the communication must be authenticated and encrypted. The designer has to choose between RSA and Elliptic Curve Cryptography (ECC) for the public key part of the system, and the choice must respect the limited resources of the devices.

A. Analyze the constraints of IoT devices

IoT devices are very different from servers or smartphones. The main constraints that affect the choice of a cryptographic algorithm are listed below.

- **Limited processing power:** Most devices use 8-bit, 16-bit or low-speed 32-bit microcontrollers running at a few MHz to about 100 MHz, often without hardware support for large-number arithmetic. Heavy operations such as RSA-2048 private key calculations can take seconds.
- **Limited memory:** Devices may have only a few KB of RAM and tens or hundreds of KB of flash. Large keys, certificates and big-integer buffers do not fit easily.
- **Limited energy:** Many sensors run on small batteries or energy harvesting for years. Every extra computation and every extra transmitted byte reduces battery life.
- **Limited bandwidth and small packets:** Wireless links such as Zigbee, Bluetooth Low Energy and LoRa carry small frames (IEEE 802.15.4 frames are about 127 bytes). Large keys and signatures must be fragmented, which increases delay and energy use.
- **Real-time response:** Locks, alarms and switches must react quickly, so slow handshakes are not acceptable.
- **Physical exposure:** Devices are installed in homes and outdoor places where an attacker can touch them, so they are exposed to tampering and side-channel attacks such as timing and power analysis.
- **Large scale and long life:** A home may have dozens of devices that stay in use for ten years or more, so key provisioning, key updates and low cost per device are important.
- **Weak randomness:** Small devices often lack a good entropy source, and generating keys on the device must be cheap and safe.

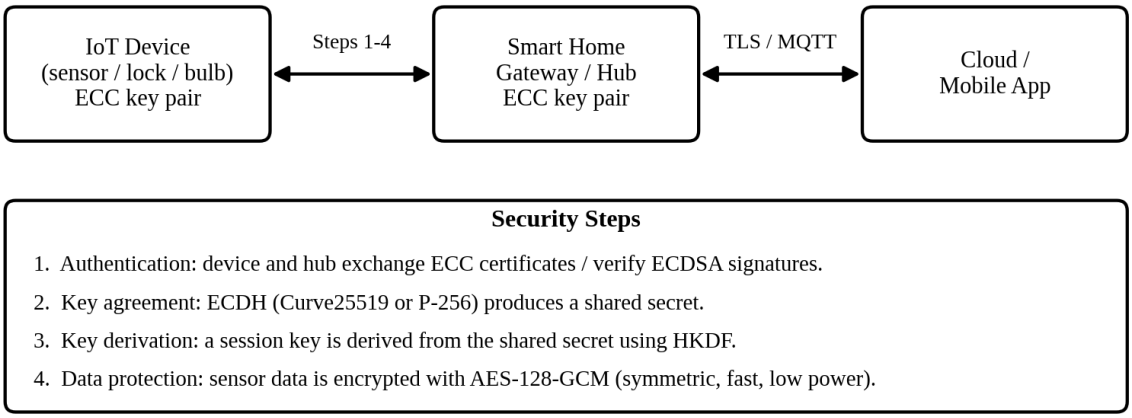
The table below shows typical classes of constrained devices (as described in RFC 7228). Most smart home sensors belong to Class 0 or Class 1.

Device class	Approximate RAM	Approximate flash / code space
Class 0	Much less than 10 KB	Much less than 100 KB
Class 1	About 10 KB	About 100 KB
Class 2	About 50 KB	About 250 KB

B. Recommend a suitable cryptographic approach and justify your answer

Recommendation: Use ECC (for example Curve25519 or NIST P-256) for authentication and key agreement, combined with a lightweight symmetric cipher such as AES-128-GCM for the actual data. ECDH is used to agree on a shared secret, ECDSA or Ed25519 is used for signatures and device identity, and the shared secret is turned into a session key that encrypts the sensor data. This hybrid design is shown in Figure 3.

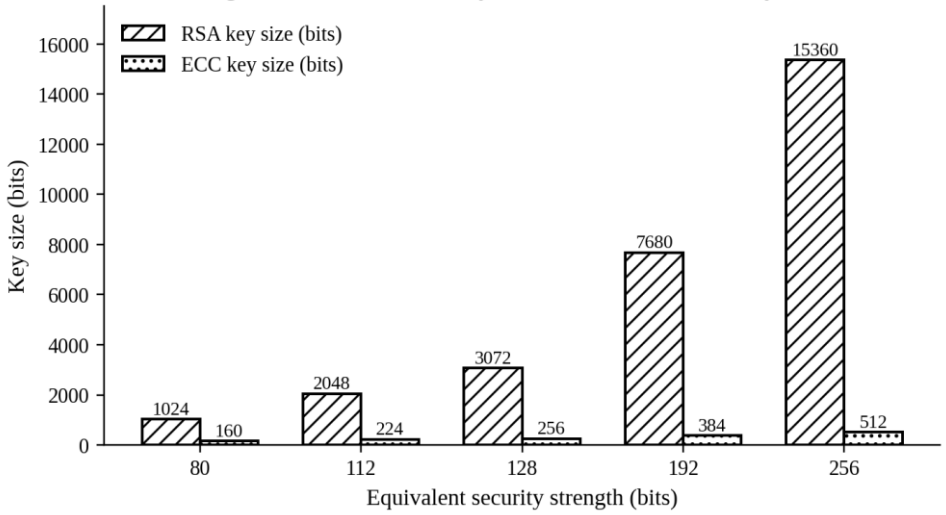
Figure 3: Secure Smart Home Communication using ECC



Justification

- **Much smaller keys:** ECC gives the same security as RSA with far shorter keys. A 256-bit ECC key is roughly as strong as a 3072-bit RSA key (Figure 4). Smaller keys need less memory and storage.
- **Lower computation and energy:** Key generation and signing are much faster in ECC, and the private key operation uses less energy, which extends battery life.
- **Less bandwidth:** An ECC public key is about 32 to 64 bytes and a signature about 64 bytes, while RSA-3072 needs 384 bytes for each. Messages fit into small wireless frames without fragmentation.
- **Faster handshakes:** Smaller numbers give quicker key agreement, so locks and alarms respond in real time.
- **Good support:** ECC is supported by standard libraries (mbed TLS, micro-ecc, wolfSSL) and by low-cost secure elements, and it is standardised by NIST and the IETF.

Figure 4: RSA vs ECC Key Size for the Same Security Level



Parameter	RSA	ECC
Key size for 128-bit security	3072 bits	256 bits
Signature size	About 384 bytes	About 64 bytes
Key generation	Slow, needs large primes	Fast
Private key operation (sign / decrypt)	Slow and power hungry	Fast and low power
Public key operation (verify / encrypt)	Fast with small exponent	Fast
Memory and bandwidth need	High	Low
Suitability for low-power IoT	Poor	Good

Points to be careful about: ECC must be implemented correctly. ECDSA needs a fresh random nonce for every signature because reusing it reveals the private key, standard well-reviewed curves should be used, and constant-time code or a hardware secure element should protect against side-channel attacks. RSA can still be useful in cases where a device only verifies signatures, for example checking a signed firmware update, because RSA verification is fast. Both RSA and ECC can be broken by large quantum computers in the future, so the design should allow algorithms to be replaced later.

Conclusion: Because IoT devices have limited CPU power, memory, battery and bandwidth, ECC is the better choice than RSA for the smart home system. It delivers the same level of security with smaller keys, faster operations and lower energy use, and combined with AES for the data it gives secure and efficient communication.